

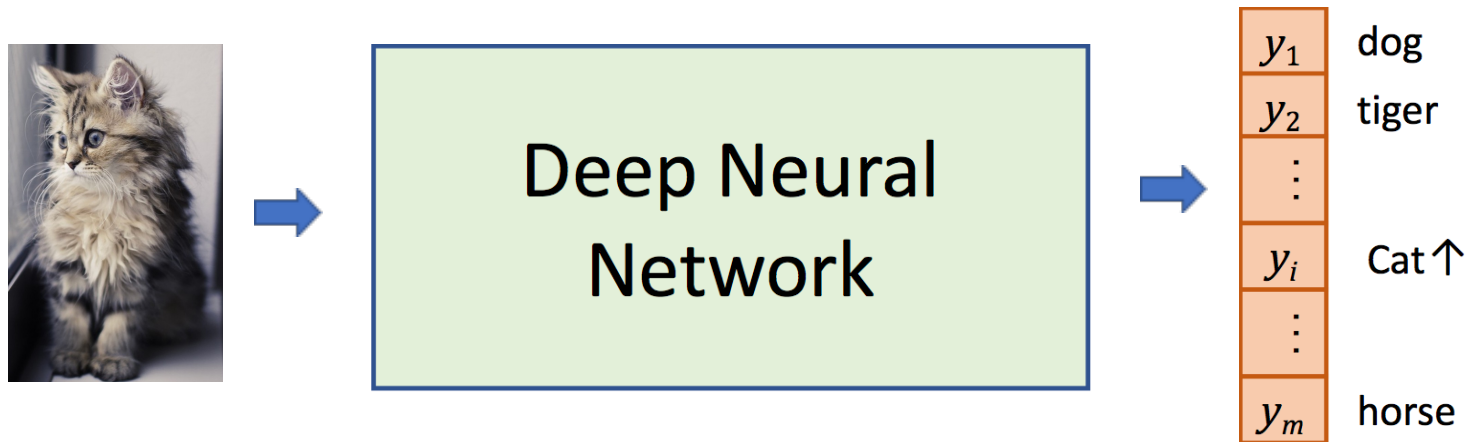
Convolutional Neural Networks (CNNs)

Jun Hong

Fully-connected neural networks (ANNs)

So far, we have learned:

- Neural networks with *fully-connected layers*
- The optimisation process to train the network/model



Each node in a hidden layer is connected to every node in the previous layer, with two functions applied: an aggregation function and an activation function.

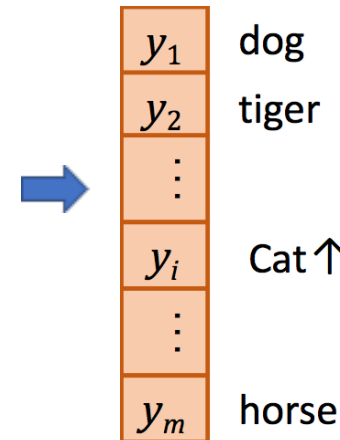
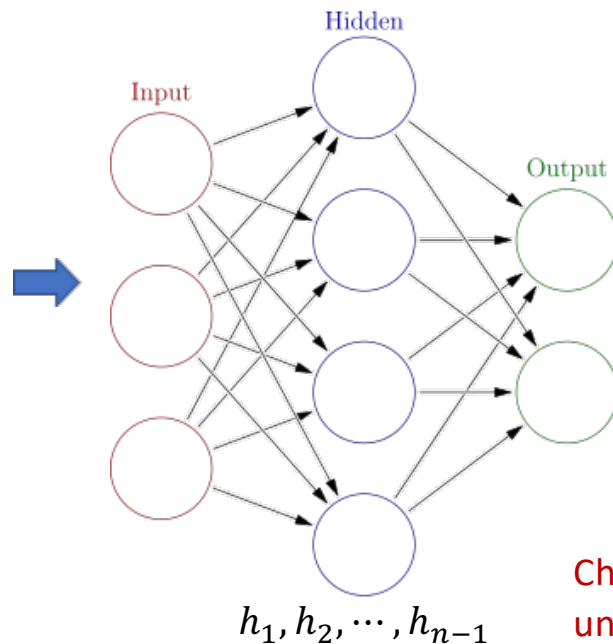
Fully-connected neural networks (ANNs)

- The input to a node in the next layer is from *all the nodes* in the previous layer so all these nodes are *looked at together globally rather than being analysed locally*, e.g. a subset of neighbouring nodes which may represent a *smaller spatial structure* as part of a bigger structure
- Computationally this leads to a huge number of weights in the network with multiple hidden layers (in the order of $O(n \times d^2)$, and with the theoretical expressive power (number of neurons) of any FCNN to approximate any continuous function estimated in the order of $O(2^n)$)

Grid-like
instead of
vector-like



Raw data



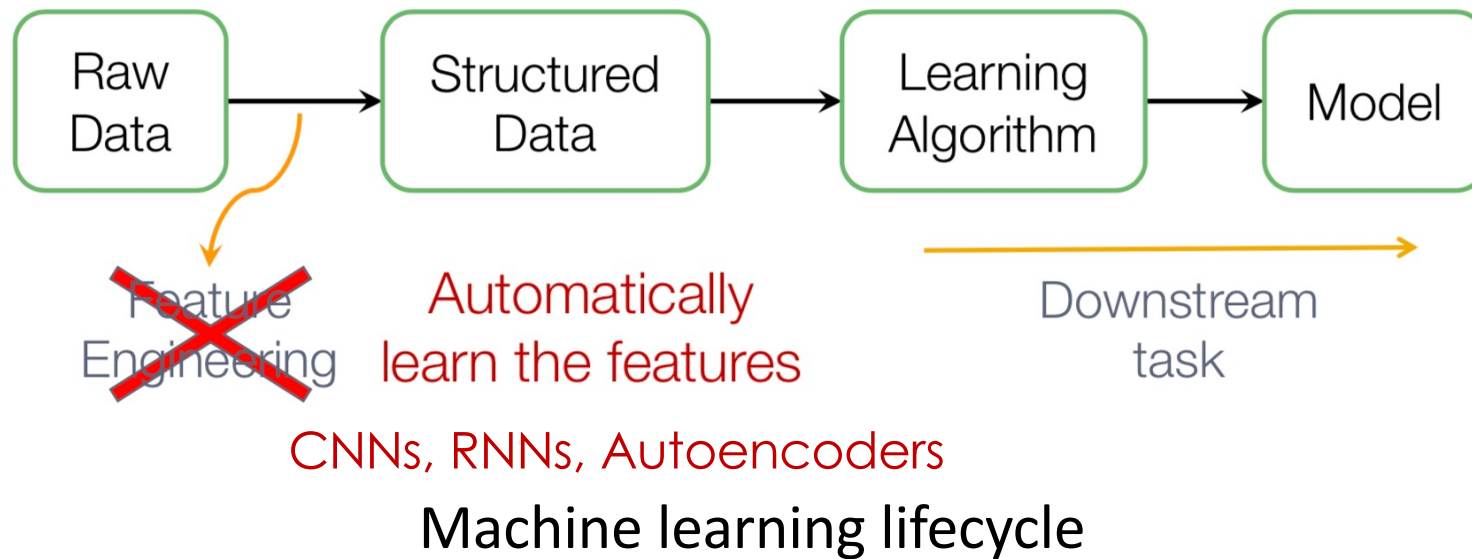
For an image (100×100 pixels), each node on the next layer, has 10,000 weights.

Think of the Wheat and chessboard problem - $2^{64} - 1$ grains to be placed upon each square!

Challenges: global connectivity, too many variances, unshared weights among variances (e.g. ImageNet)

Feature learning in deep learning

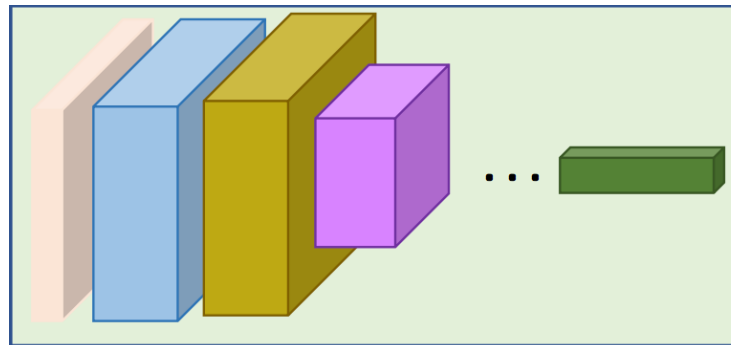
Goal: Efficient task-independent feature learning



Convolutional neural networks (CNN)

A convolutional neural network is an architecture consisting of *extra layers for feature learning*, in addition to fully-connected layers, *which play different roles from the FC layers*

- Input
- Convolutional
- Activation (ReLU)
- Pooling
- Fully-connected (FC)
- Drop-out
- Batch normalisation (BN)



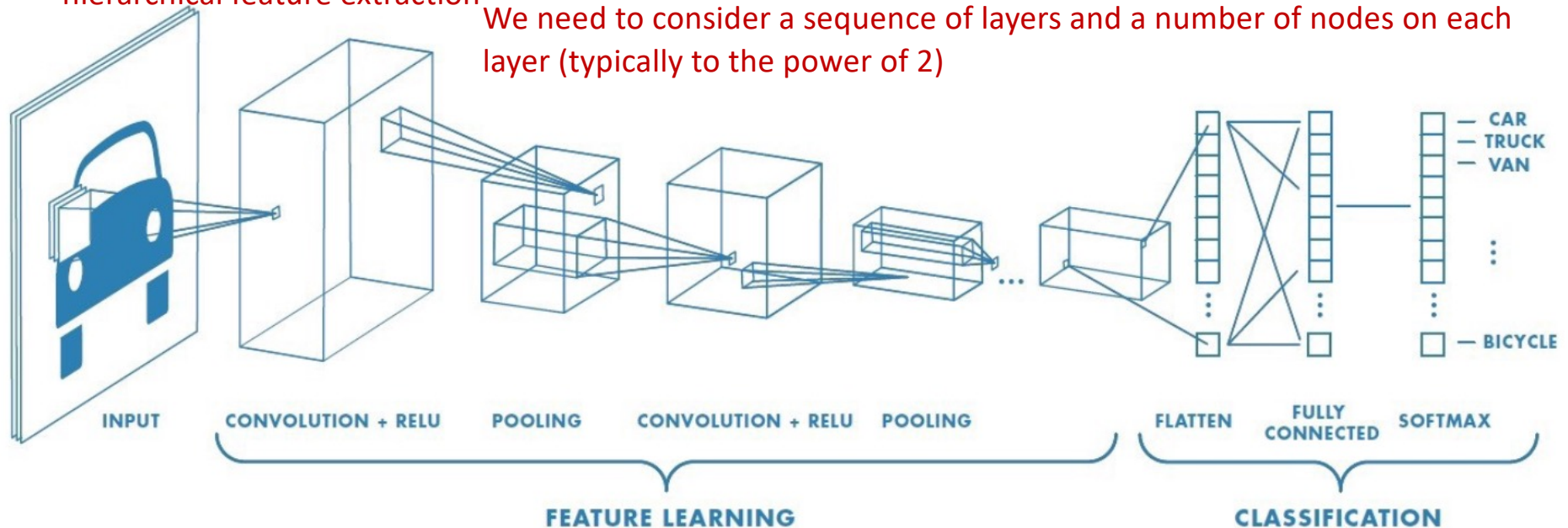
y_1	dog
y_2	tiger
$\vdots$	
y_i	Cat $\uparrow$
$\vdots$	
y_m	horse

CNNs (AlexNet, 2012) revolutionised AI - dramatically reduce numbers of weights and nodes by leveraging local connectivity and weight sharing.

Convolutional neural networks (CNN)

Advantages: local feature learning, parameter sharing, invariances (e.g. translation, rotation, size) and hierarchical feature extraction

We need to consider a sequence of layers and a number of nodes on each layer (typically to the power of 2)



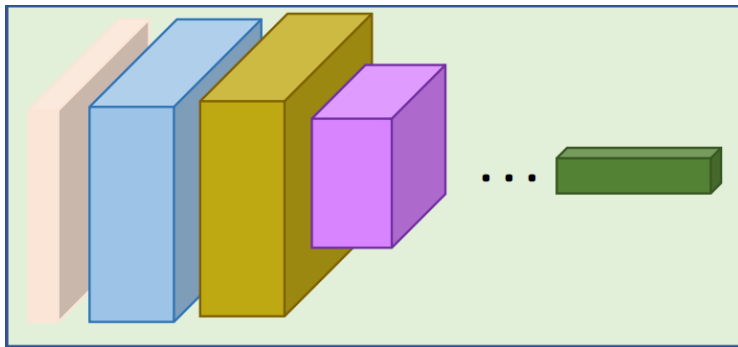
Two layers of convolution plus two activation and two pooling layers used for feature learning
Hierarchical feature extraction: Feature maps become more abstract as we go deeper

Convolutional neural networks (CNN)

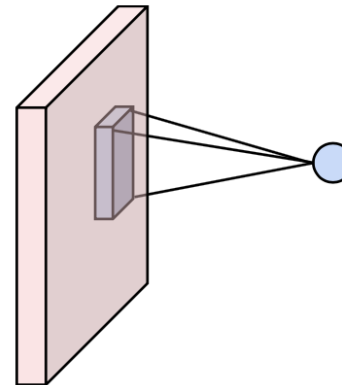
Convolutional layer – the main building block

- It slides a *smaller learnable filter* over the input data to extract *localised spatial features* from a *subset of neighbouring nodes* (kind of local aggregation/abstraction) into the next layer

Different layers in a CNN



Convolution Layers

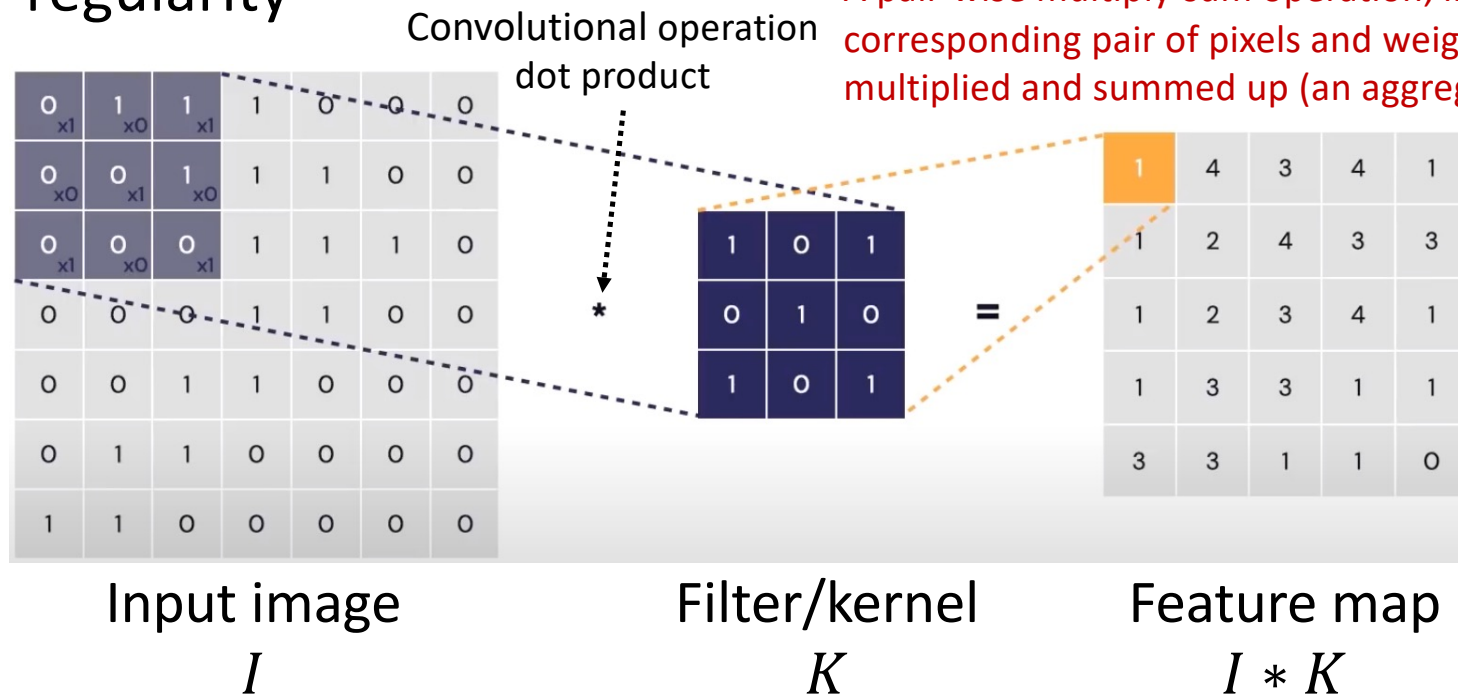


Local feature learning to detect local patterns (edges, textures, shapes)

Convolution on images

Extract features representing localised spatial regularity

The filter is **applied** to a local area of the image, and **the dot product** between the area and the filter is calculated



A pair-wise multiply-sum operation, i.e., each corresponding pair of pixels and weights are multiplied and summed up (an aggregation function)

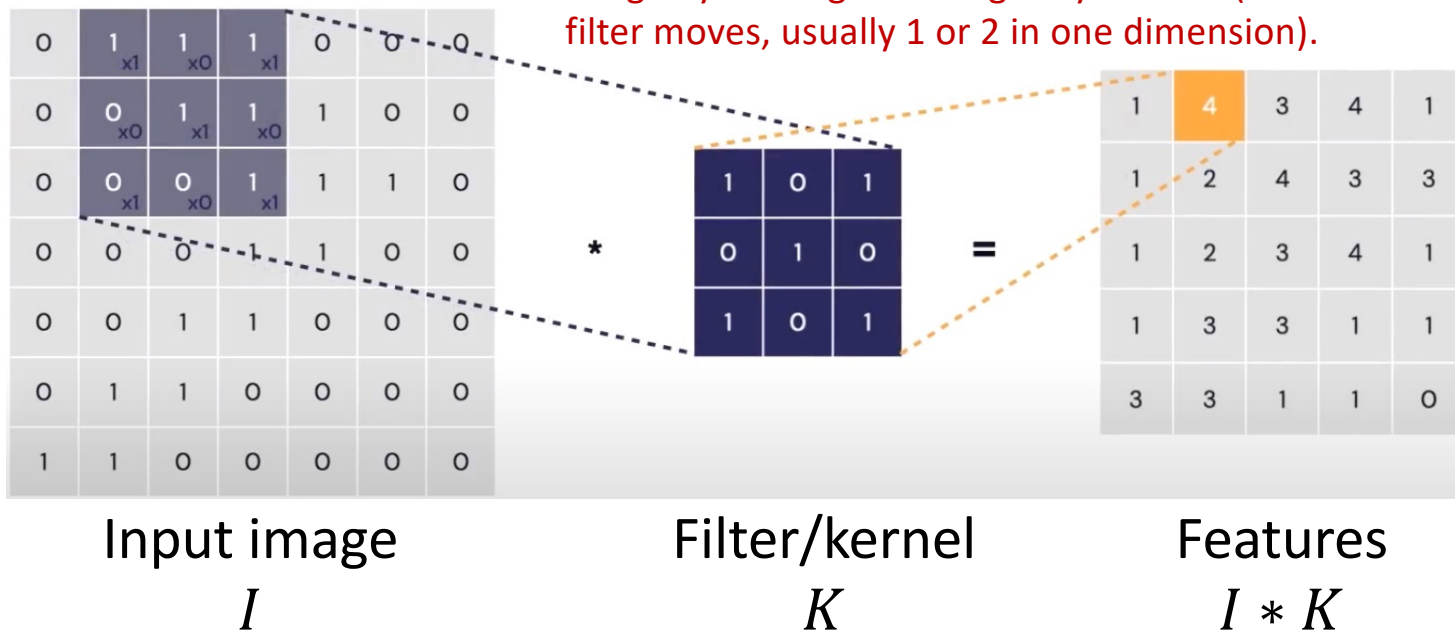
What localized spatial feature is detected by this filter?

Parameter sharing: A learnable, sharable matrix of weights

Convolution on images

Extract features representing localised spatial regularity

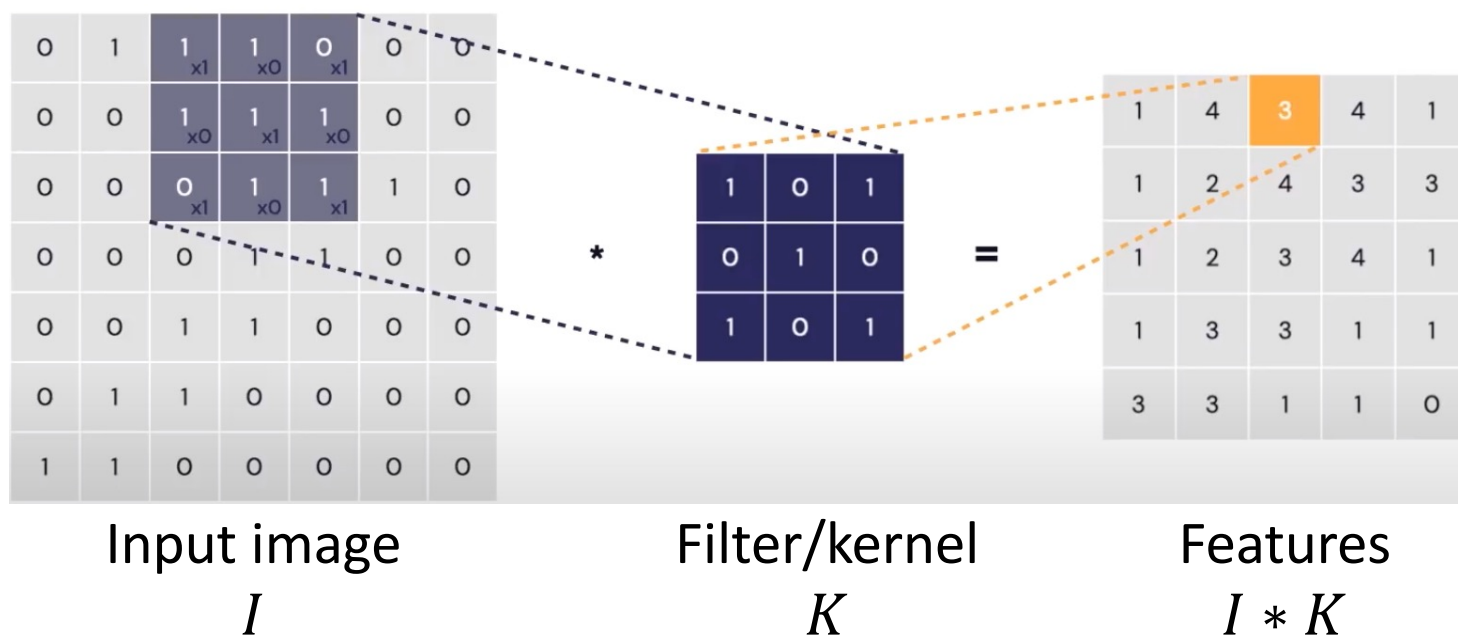
The filter **is then applied** to the next local area of the image by shifting to the right by a stride (how much the filter moves, usually 1 or 2 in one dimension).



Convolution on images

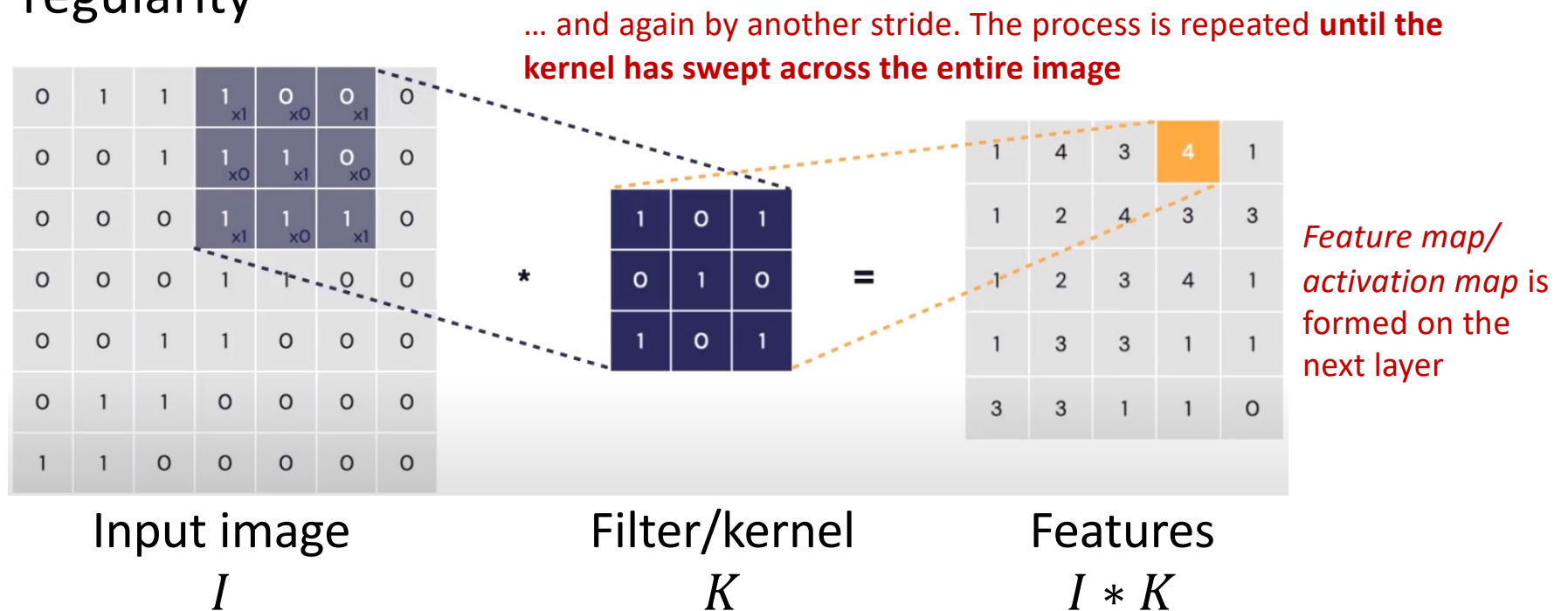
Extract features representing localised spatial regularity

The filter shifts again by another stride



Convolution on images

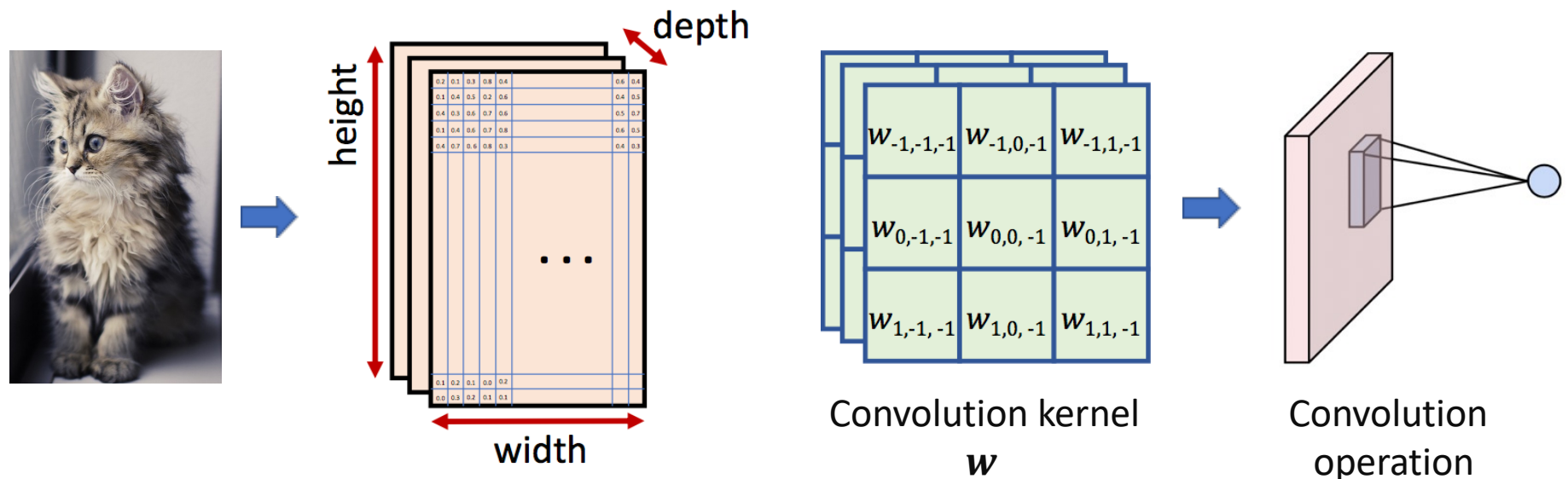
Extract features representing localised spatial regularity



Translation invariance: A cat is recognized regardless of its position in the image

Convolutional neural networks (CNN)

An example: The input data (for representing an RGB image) has a depth of 3 plus a length and a width. We apply a learnable filter with $3 \times 3 \times 3$ weights onto a $3 \times 3 \times 3$ cube in the input data (think of this as all input nodes to a node in the next layer in a *small fully-connected network*)

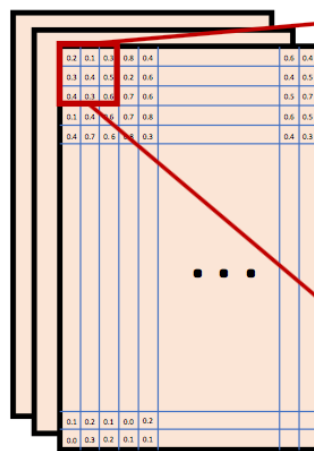


Easier to think of a depth of 1 so apply a filter of 3×3 to a square of 3×3

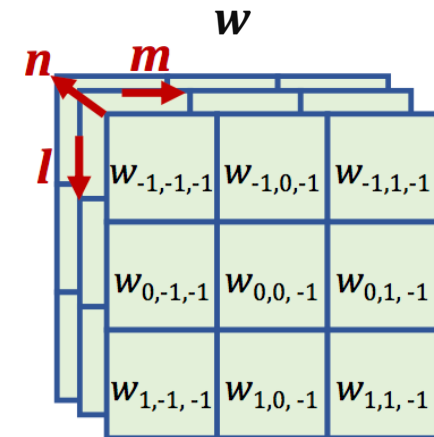
Convolutional neural networks (CNN)

The filter is applied to produce a new node in the next layer:

$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$



0.2	0.1	0.3
0.3	0.4	0.5
0.4	0.3	0.6

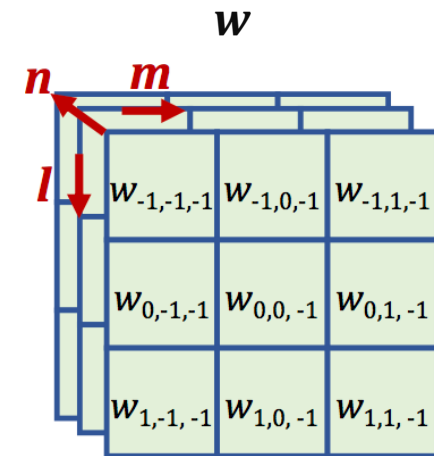
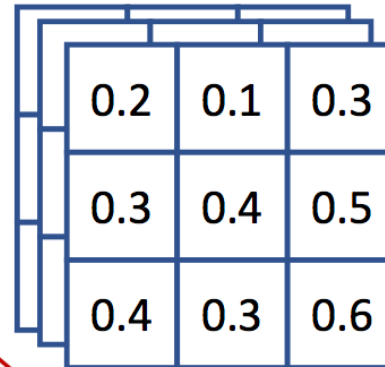
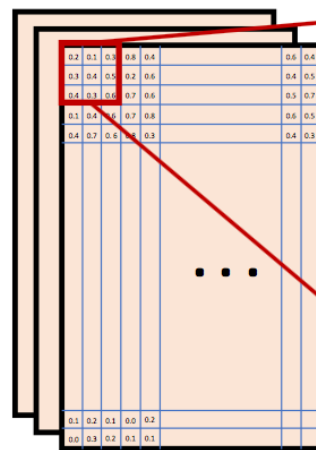


$$w_b + 0.2 \times w_{-1,-1,-1} \dots$$

Convolutional neural networks (CNN)

The filter starts with the top-left corner of the input data, repeatedly slides over every pixel, forming a new node in the next layer each time, until the whole input data is covered:

$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$

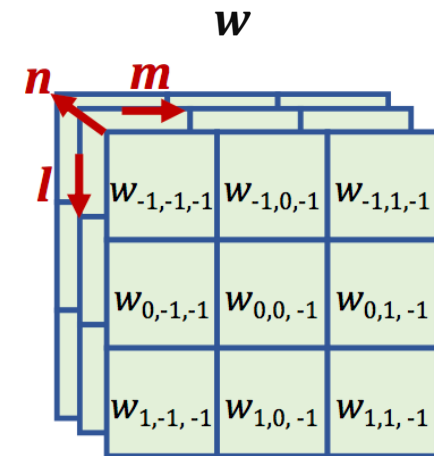
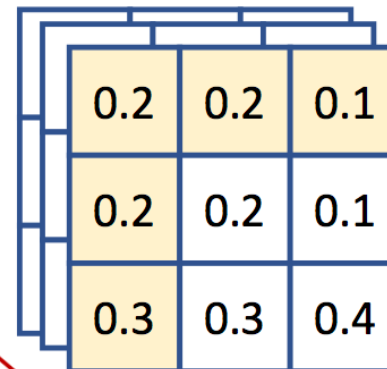
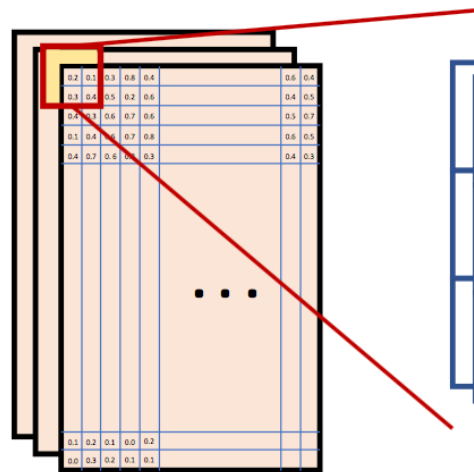


$$w_b + 0.2 \times w_{-1,-1,-1} \dots$$

Convolutional neural networks (CNN)

The input data is *padded* for the bordering pixels (by *duplicating the pixels/nodes*) so that they can be the centre of a neighbouring subarea:

$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$

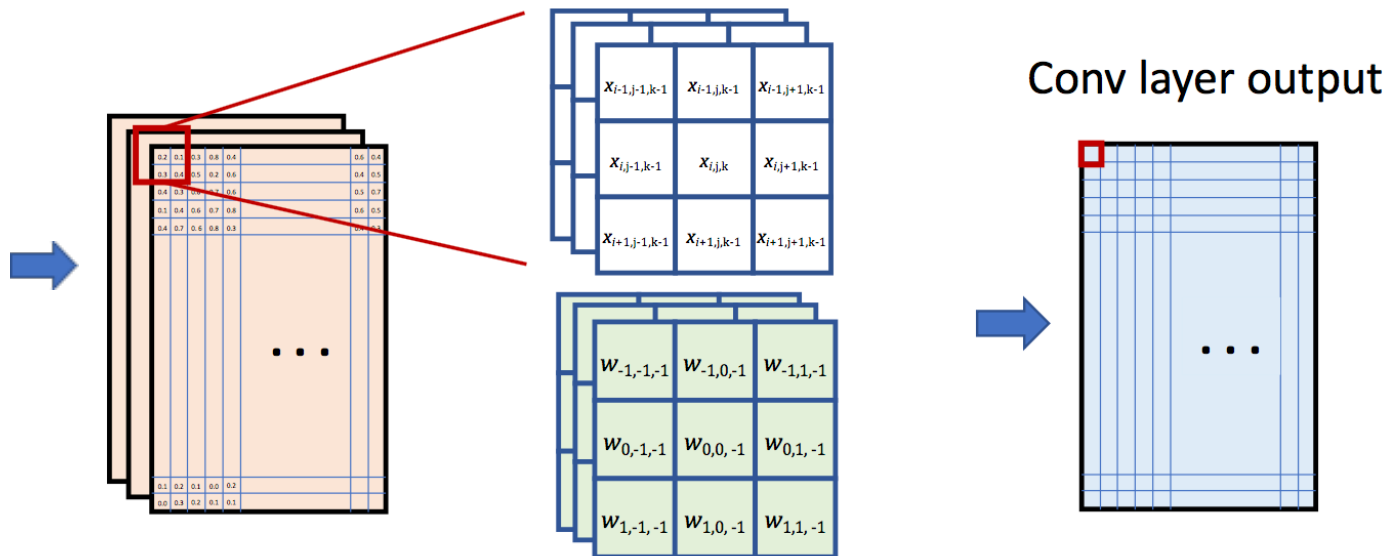


$$w_b + 0.2 \times w_{-1,-1,-1} \dots$$

Convolutional neural networks (CNN)

Calculating the convolutional layer:

$$w_b + \sum_{l,m,n=-1}^1 x_{i+l,j+m,k+n} w_{l,m,n}$$



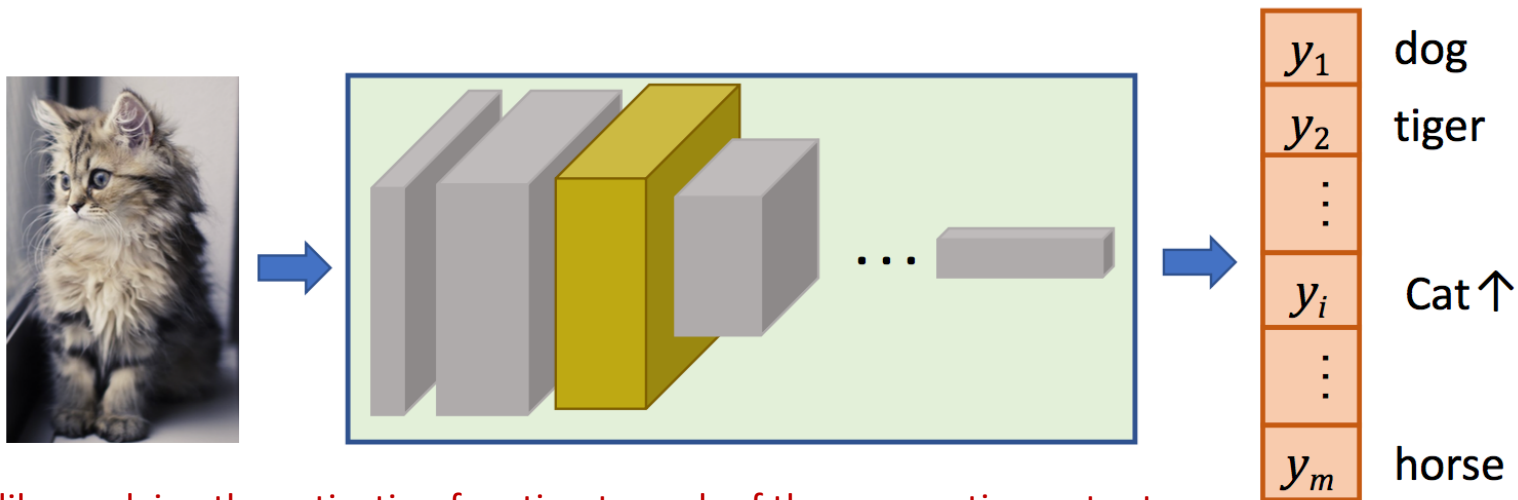
This is like applying the aggregation function repeatedly to each of the filtered areas to produce the weighted sums.

Convolutional neural networks (CNN)

Activation layer

- It represents *non-linearity* in the network
- Non-linearity helps the network learn complex patterns
- The most common activation functions: Sigmoid, ReLU and Tanh

Applies an activation function to each node in the feature map/activation map



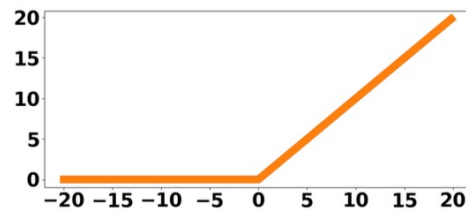
This is like applying the activation function to each of the aggregation outputs.

Convolutional neural networks (CNN)

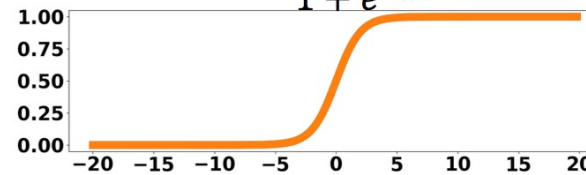
Activation layer

$$\text{ReLU}(x) = \max(0, x)$$

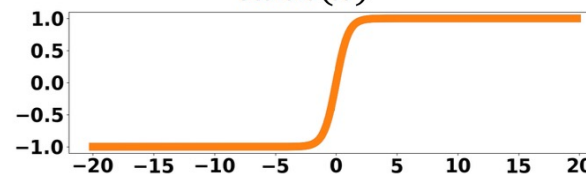
(Rectified Linear Unit)



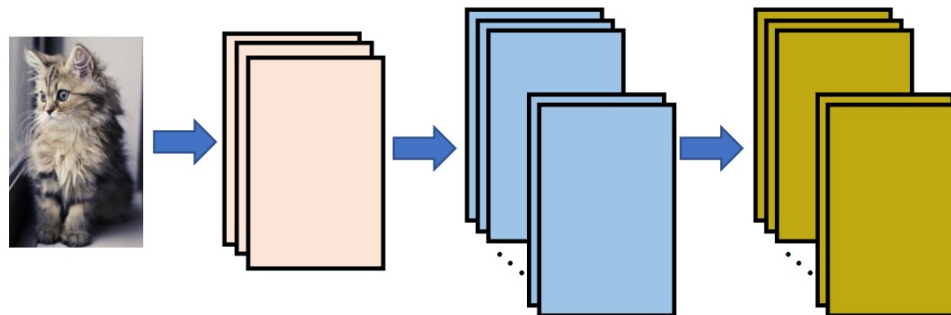
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



$$\tanh(x)$$



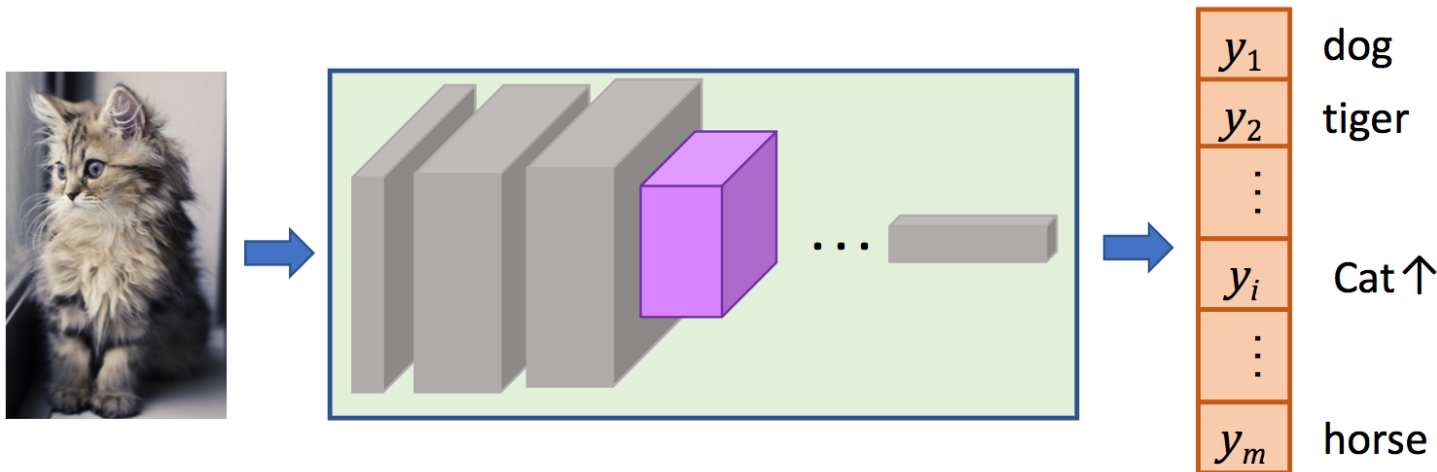
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



Convolutional neural networks (CNN)

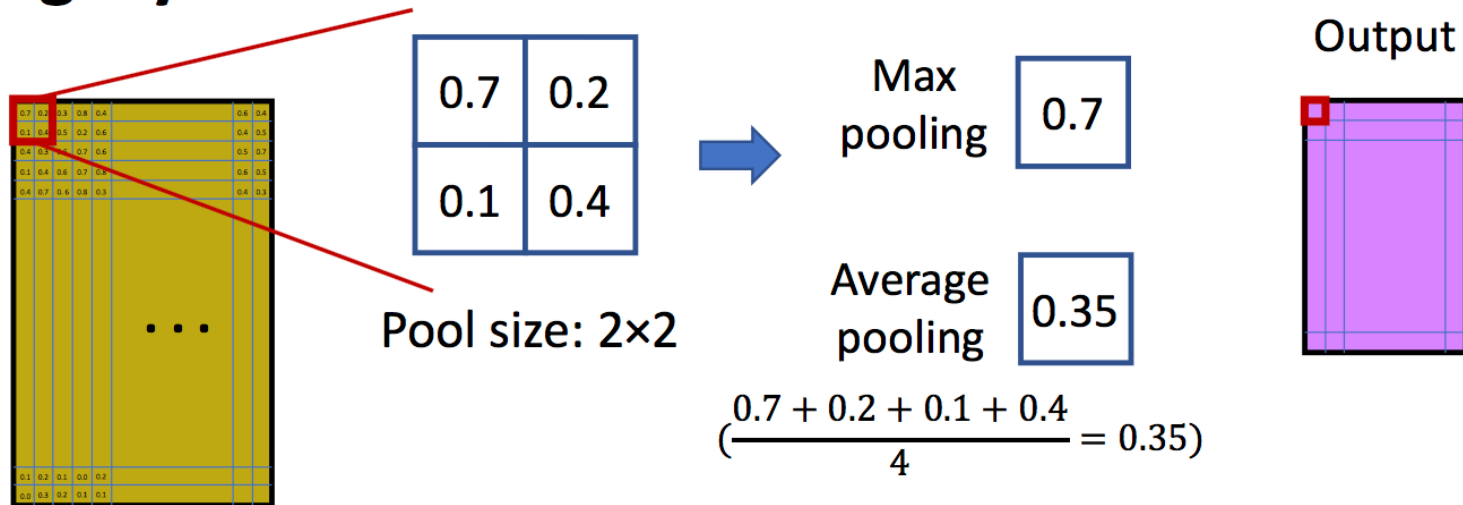
Pooling layer

- It down-samples the input data (reduce the number of nodes by an integer factor)
- Reduces the network sensitivity to the locations of features
- Common pooling functions: *average and max pooling* (compared to a convolutional operation which is an aggregation; However, the average or max function is not weighted!)



Convolutional neural networks (CNN)

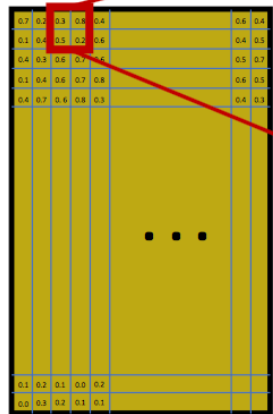
Pooling layer



This reduces the number of nodes in the next layer by an integer factor, e.g. a factor of 4 in this case.

Convolutional neural networks (CNN)

Pooling layer



0.3	0.8
0.5	0.2

Pool size: 2×2
Stride: 2



Max
pooling

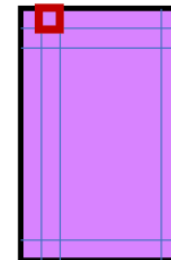
0.8

Average
pooling

0.45

$$\left(\frac{0.3 + 0.8 + 0.5 + 0.2}{4} = 0.45 \right)$$

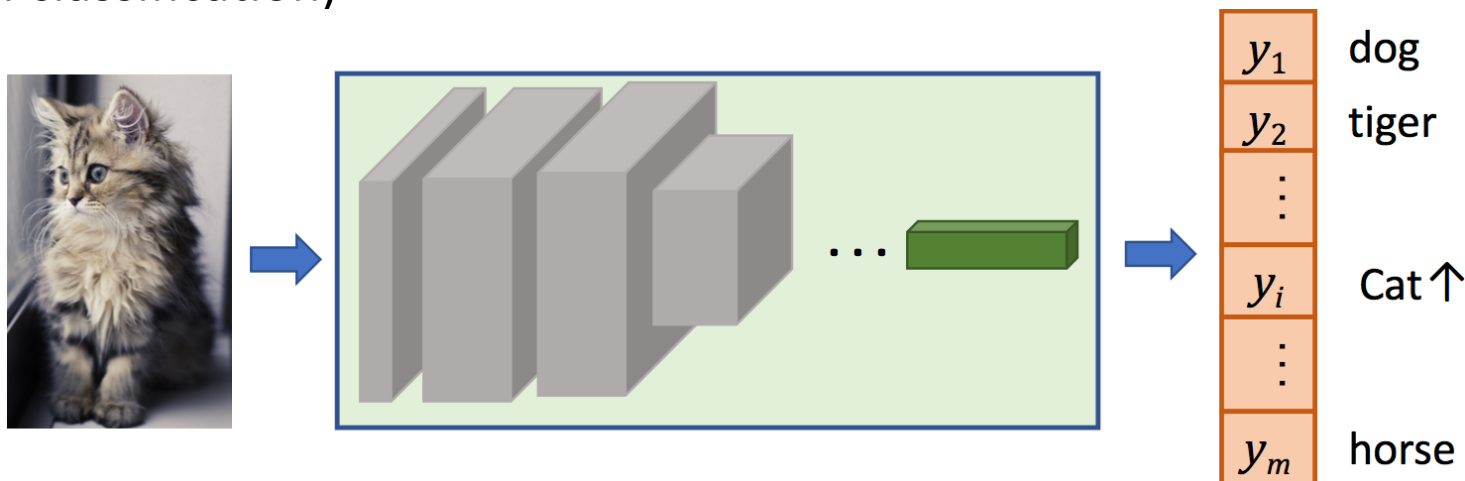
Output



Convolutional neural networks (CNN)

Fully-connected layer

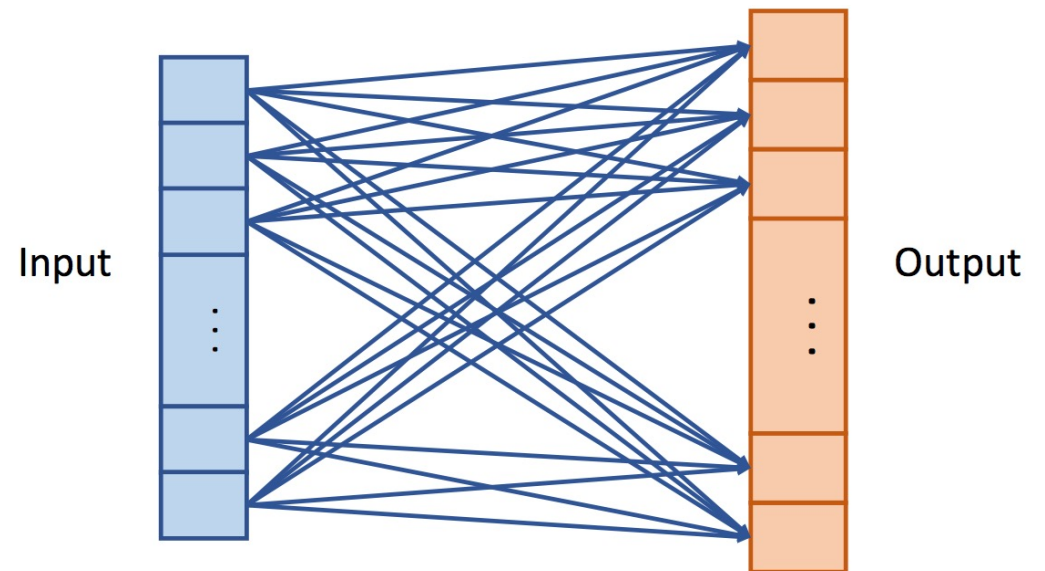
- It connects all the input nodes to every node in the next layer
- Usually used as the last layer in the network before the output layer (e.g. classification)



Convolutional neural networks (CNN)

Fully-connected layer

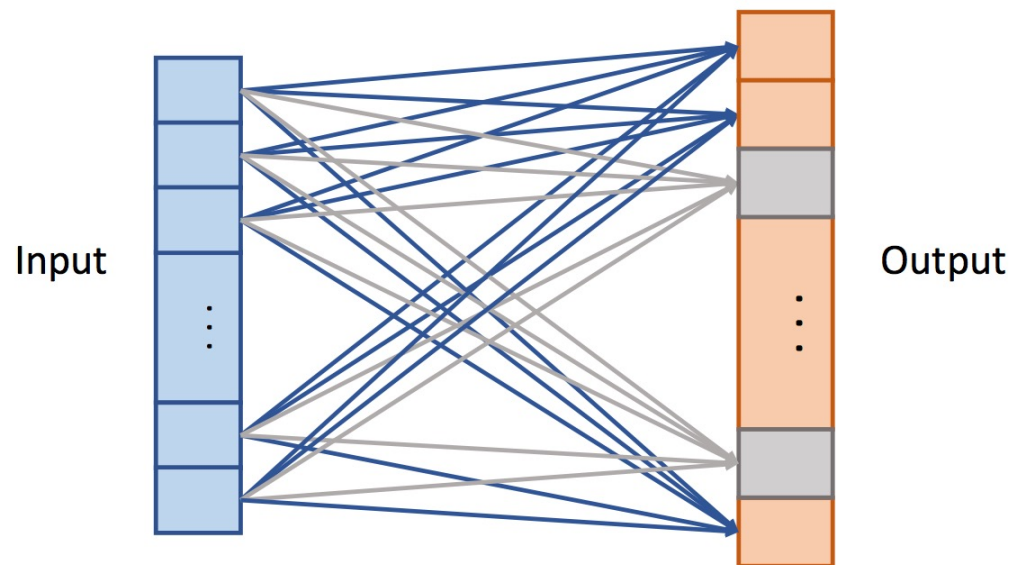
- It connects all the input nodes to every node in the next layer
- Usually used as the last layer in the network before the output layer (e.g. classification)
- This is the same as we have in a fully-connected neural network



Convolutional neural networks (CNN)

Drop-out layer

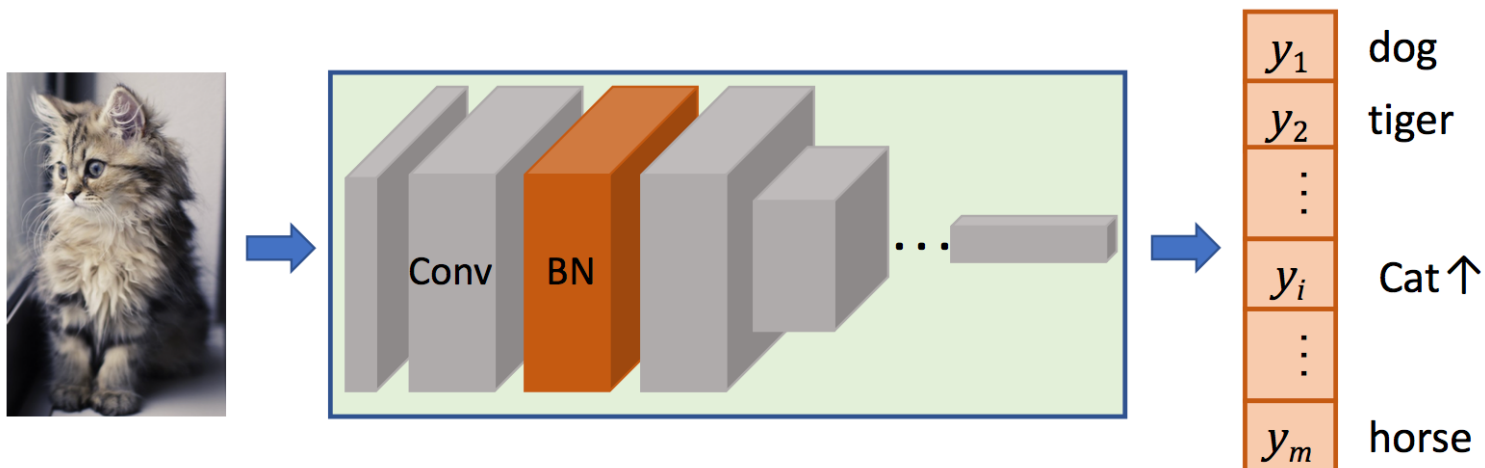
- It *randomly drops out* (by setting weights to 0, i.e. applying a mask/matrix of 0s and 1s) input nodes during training
- Helps prevent overfitting (regularisation, i.e. discounting the dependency on the input data to a certain extent)



Convolutional neural networks (CNN)

Batch-normalisation layer

- It normalises the input data across *mini-batches* (the size of mini-batches is a hyperparameter)
- Helps stabilise the network against the variances in input data distributions
- Helps better generalise the trained network to the test data (regularisation)



Convolutional neural networks (CNN)

Batch-normalisation layer

It first subtracts the mini batch mean (μ) from the the input data (x) and divides it the mini-batch standard deviation (σ)

$$\hat{x} = \frac{x - \mu}{\sigma}$$

*$\hat{x}$ will have a *standard normal distribution* with zero mean ($\mu = 0$) and unit standard derivation ($\sigma = 1$)*

This is then shifted and scaled by two learnable parameters, γ and β

$$x^{BN} = \hat{x} \cdot \gamma + \beta$$

The BN output has a distribution with mean β and standard deviation γ

Summary

- We have mainly discussed CNN, in particular the different layers in CNN
 - Convolutional, activation, pooling, fully-connected, drop-out and batch-normalisation layers
- Our main focus in this module is on CNN due to time limitation
- There are a number of other popular deep neural networks that can be further explored if needed
 - RNN
 - GAN
- More may be covered in a final-year module

Resources

- Stanford University School of Engineering
 - Lecture 1 | Introduction to Convolutional Neural Networks for Visual Recognition
 - <https://www.youtube.com/watch?v=vT1JzLTH4G4>
 - Even though this is a course for visual recognition, it covers a lot of fundamentals of neural networks and deep learning in the context of computer vision.
 - There are a lot of visual illustrations/images that actually help better understand the fundamental concepts of deep learning.
 - The two young academics teaching the course are very good, with very clear presentations.

Resources

- Deep Learning for Computer Vision
 - by Justin Johnson from University of Michigan, formerly a PhD student in Computer Vision Lab at Stanford
 - <https://www.youtube.com/watch?v=dJYGatp4SvA&list=PL5-TkQAfAZFbzxjBHtzdVCWE0Zbhomg7r&index=1>

Resources

- MIT 6.S191 (2020): Convolutional Neural Networks
 - by Alexander Amini from MIT as part of MIT Introduction to Deep Learning 6.S191
 - <https://www.youtube.com/watch?v=iaSUYvmCekI>
 - Very clear explanations of the motivations, differences using intuitive examples